

Software Requirements Specification

Project:	Agentic AI & Privacy Browser
Author:	Ponduru Sampath Kumar
Version:	2.0
Date:	September 15, 2026
Status:	Under Review

Revision History

Version	Date	Author	Description
1.0	2026-09-01	Ponduru Sampath Kumar	Initial draft of the SRS document.
2.0	2026-09-15	Ponduru Sampath Kumar	Added glossary, assumptions, use cases, external interfaces, data requirements, error handling, deployment sections. Enhanced all functional and non-functional requirements with measurable criteria and MoSCoW priority labels.

Table of Contents

1. Introduction

1.1 Purpose

1.2 Scope

1.3 Glossary & Definitions

1.4 References

1.5 Priority Scheme

2. Overall Description

2.1 Product Perspective

- 2.2 User Classes & Characteristics
- 2.3 Assumptions & Dependencies
- 2.4 Constraints
- 3. Use Cases
- 4. Functional Requirements (FR)
 - 4.1 Core Browsing Functionality
 - 4.2 Privacy & Ad-Blocking Engine
 - 4.3 Agentic AI Assistant (Groq Powered)
- 5. Non-Functional Requirements (NFR)
 - 5.1 Performance & Resource Efficiency
 - 5.2 Security & Safety
 - 5.3 Reliability & Availability
 - 5.4 Maintainability
 - 5.5 Accessibility
- 6. External Interface Requirements
- 7. Data Requirements
- 8. Error Handling & Fallback Behavior
- 9. Deployment & Installation
- 10. Appendices

1. Introduction

1.1 Purpose

This document specifies the software requirements for a new desktop web browser. The application combines aggressive privacy protections (similar to Brave) with an autonomous, agentic AI layer, capable of executing multi-step workflows on behalf of the user. This SRS serves as the authoritative agreement between stakeholders and the development team for all features and constraints in version 1.0.

1.2 Scope

The browser is designed as a lightweight desktop application to accommodate development on mid-range hardware. It will utilize a lightweight framework (e.g., Tauri with Rust) to hook

into native system webviews, relying on `adblock-rs` for network filtering and the Groq API for Large Language Models (LLMs) to power the agentic reasoning layer.

The following items are **explicitly out of scope** for version 1.0:

- Mobile platforms (Android, iOS).
- Browser extension / add-on ecosystem.
- Cross-device bookmark and history synchronization.
- Built-in VPN or Tor integration.
- PDF viewer and media player.

1.3 Glossary & Definitions

Term	Definition
Accessibility Tree	A simplified, structured representation of a web page's interactive elements (buttons, links, inputs, headings) derived from the DOM, used to provide context to the AI agent.
<code>adblock-rs</code>	An open-source Rust library that parses and evaluates ad-blocking filter lists (EasyList, uBlock Origin syntax) to determine whether a network request should be blocked.
Agentic Reasoning Loop	The cyclic process of: (1) extracting page state, (2) sending it to an LLM, (3) receiving a structured action, (4) executing the action in the browser, and (5) observing the result.
Cosmetic Filtering	The practice of injecting CSS rules into a web page to hide DOM elements (e.g., ad containers) that passed network-level filtering.
DOM (Document Object Model)	The tree-structured representation of an HTML document that the browser uses for rendering and scripting.
Groq API	A cloud-hosted inference API powered by Groq's Language Processing Unit (LPU) hardware, providing ultra-low-latency LLM inference.
IPC (Inter-Process Communication)	The mechanism by which the Tauri frontend (JavaScript) communicates with the Rust backend through command invocations and event listeners.
LPU (Language Processing Unit)	Groq's custom silicon chip architecture optimized for sequential token generation in large language models, offering significantly lower latency than GPU-based inference.

- **[SHOULD]** — Important and expected, but the product can launch with a workaround if necessary.
- **[COULD]** — Desirable enhancement. Implement if time and resources permit.

2. Overall Description

2.1 Product Perspective

The browser consists of three primary subsystems:

- **UI & Rendering Shell:** A lightweight web-tech frontend running inside a Tauri window, using the OS-native webview (WebView2 on Windows, WebKit on macOS/Linux) for page rendering.
- **Privacy Engine:** A Rust-based backend utilizing `adblock-rs` to intercept web requests and block trackers before they load. It operates as a local filter between the webview and the network layer.
- **Agentic Pipeline:** A background service that extracts the web page's Document Object Model (DOM), simplifies it into an accessibility tree, and passes it to an LLM via the Groq API to generate actionable browser commands.

2.2 User Classes & Characteristics

User Class	Description	Technical Proficiency
Privacy-Conscious General User	Users who want built-in ad-blocking and tracker protection without needing to install extensions. They expect a familiar browsing experience with enhanced privacy out-of-the-box.	Low to Medium
Power User / Developer	Users who will leverage the agentic AI assistant for complex, multi-step web automation tasks (e.g., data extraction, form filling workflows, research tasks). They are comfortable providing natural language instructions and reviewing agent actions.	Medium to High
Accessibility User	Users who rely on keyboard navigation, screen readers, or other assistive technologies. The agentic AI layer may serve as an additional accessibility aid for web interaction.	Variable

2.3 Assumptions & Dependencies

- **A-1:** The target operating system has a compatible native webview installed (WebView2 on Windows 10+, WebKit on macOS 11+ / Linux with WebKitGTK).
- **A-2:** The user has a stable internet connection for AI-powered features. AI features degrade gracefully when offline (see Section 8).
- **A-3:** The Groq API remains commercially available with a free or affordable tier that supports the required models. A vendor change would require re-integration effort.
- **A-4:** The `adblock-rs` library continues to be maintained and compatible with current EasyList and uBlock Origin filter syntax updates.
- **A-5:** Community-maintained filter lists (EasyList, EasyPrivacy, Disconnect.me) continue to be updated by their respective maintainers.
- **A-6:** The Tauri framework maintains backward compatibility within its v2.x release line.

2.4 Constraints

- **C-1 Hardware:** The codebase must compile cleanly on a system with 8–16 GB of RAM. Forking and compiling the core Chromium source repository is strictly prohibited.
- **C-2 AI Compute:** Heavy reasoning tasks must be offloaded to cloud APIs (Groq) to prevent system freezing. No local LLM inference is permitted in v1.0.
- **C-3 Webview Limitations:** The application is constrained by the capabilities of the OS-native webview. Features requiring direct Chromium DevTools Protocol access may not be available on all platforms.
- **C-4 Single Developer:** The initial development is undertaken by a single developer; scope must remain feasible for individual execution.

3. Use Cases

UC-1: Private Web Browsing

Actor: Privacy-Conscious General User

Precondition: Browser is installed and launched.

Flow:

1. User types a URL or search query into the address bar.
2. The Privacy Engine intercepts all outbound requests and evaluates them against filter lists.

3. Blocked trackers and ads are silently dropped; cosmetic filters hide residual ad containers.
4. The page loads with a visible shield icon showing the count of blocked items.
5. User can click the shield icon to see a breakdown of blocked trackers, ads, and fingerprinting scripts.

Postcondition: The page renders without ads or trackers. No tracking cookies are stored.

UC-2: AI-Assisted Form Filling

Actor: Power User

Precondition: User is on a web page with a multi-field form. Groq API is reachable.

Flow:

1. User opens the AI assistant panel and types: *"Fill out this job application with my resume data."*
2. The system extracts the accessibility tree of the current page.
3. The accessibility tree and user instruction are sent to the Groq API (Llama 3.3 70B).
4. The LLM returns a sequence of structured actions (type into field X, select option Y).
5. Each action is displayed in the action log panel before execution.
6. The agent executes each action sequentially, pausing for user confirmation on sensitive fields (e.g., SSN, salary).

Postcondition: The form is filled. The user reviews and submits manually.

UC-3: Multi-Step Web Research

Actor: Power User

Precondition: User is on a search engine. Groq API is reachable.

Flow:

1. User types: *"Find the top 5 open-source Rust web frameworks, compare their GitHub stars and last commit date, and summarize the results."*
2. The agent navigates to search results, opens each framework's GitHub page in sequence.
3. For each page, the agent extracts the relevant data (stars, last commit) from the accessibility tree.

4. After collecting all data, the agent compiles a summary and presents it in the assistant panel.
5. Each navigation step is logged in the action panel with status indicators.

Postcondition: A structured comparison is displayed in the assistant panel.

UC-4: Offline / Degraded Mode Browsing

Actor: Any User

Precondition: Groq API is unreachable (network issue, rate limit, API outage).

Flow:

1. User launches the browser or the API connection drops mid-session.
2. The browser detects the API is unreachable and displays a non-intrusive notification:
"AI assistant is temporarily unavailable."
3. All core browsing and privacy features continue to function normally.
4. The AI assistant panel shows a disabled state with a retry button.
5. When connectivity is restored, the assistant automatically re-enables.

Postcondition: The user can continue browsing without disruption. AI features resume when connectivity is restored.

UC-5: Page Summarization

Actor: Privacy-Conscious General User

Precondition: User is reading a long article. Groq API is reachable.

Flow:

1. User clicks the "Summarize" button or types: *"Summarize this page."*
2. The system extracts the page's text content (excluding navigation, ads, footers).
3. The content is sent to the lightweight model (Llama 3.1 8B Instant) for summarization.
4. A concise summary is displayed in the assistant panel within 2 seconds.

Postcondition: The user receives a readable summary without leaving the page.

4. Functional Requirements (FR)

4.1 Core Browsing Functionality

- **FR-1.1: [MUST]** The system shall provide standard web navigation, including address bar URL entry with autocomplete suggestions, forward/backward history traversal, and page reloading (including hard reload bypassing cache).
- **FR-1.2: [MUST]** The system shall support a multi-tab interface with the ability to create, close, reorder, duplicate, and pin tabs. Inactive tabs shall be suspended after a configurable idle period (default: 5 minutes) to reclaim memory.
- **FR-1.3: [MUST]** The system shall manage user data locally, including bookmarks (with folder organization), browsing history (with search), and secure password autofill. Passwords shall be encrypted at rest using the OS-native credential manager (Windows Credential Manager / macOS Keychain / Linux Secret Service) via Tauri's keyring plugin. A master password option shall be available for additional protection.
- **FR-1.4: [SHOULD]** The system shall support file downloads with a download manager that displays progress, allows pausing/resuming, and provides a download history.
- **FR-1.5: [SHOULD]** The system shall provide a find-in-page feature (Ctrl+F / Cmd+F) with match highlighting and navigation between results.
- **FR-1.6: [SHOULD]** The system shall support standard keyboard shortcuts for all core navigation actions (new tab, close tab, reload, back, forward, address bar focus, find-in-page).
- **FR-1.7: [COULD]** The system shall provide a reading mode that strips non-essential page elements to present article content in a clean, readable format.

4.2 Privacy & Ad-Blocking Engine

- **FR-2.1 Network Interception: [MUST]** The browser shall intercept all outbound requests and evaluate them against local blocklists before transmission. Interception shall be implemented via Tauri's `on_navigation` event and, where supported, via platform-specific webview delegate methods. Known limitations of the native webview interception API shall be documented per platform.
- **FR-2.2 Rule Parsing: [MUST]** The system shall support EasyList and uBlock Origin rule syntaxes using the `adblock-rs` library. Filter lists shall be auto-updated every 24 hours on application startup, with the option for manual refresh from the settings panel.
- **FR-2.3 Cosmetic Filtering: [MUST]** The browser shall inject CSS into loaded pages to collapse and hide empty DOM elements left behind by blocked network requests. The

user shall be able to toggle cosmetic filtering on a per-site basis via the shield icon.

- **FR-2.4 Tracker Protection:** **[MUST]** The system shall block known fingerprinting scripts targeting the following vectors: canvas fingerprinting, WebGL renderer enumeration, AudioContext fingerprinting, and font enumeration. Fingerprinting definitions shall be sourced from the EasyPrivacy and Disconnect.me tracking protection lists.
- **FR-2.5 HTTPS Upgrade:** **[SHOULD]** The browser shall automatically upgrade all HTTP requests to HTTPS where a secure connection is available. The user shall be warned with a prominent indicator when visiting a site over plain HTTP.
- **FR-2.6 Cookie Management:** **[SHOULD]** The system shall provide configurable cookie policies including: block all third-party cookies (default), clear all cookies on session exit, per-site cookie whitelisting, and automatic cookie consent banner dismissal.
- **FR-2.7 Privacy Dashboard:** **[COULD]** The browser shall display a per-page privacy dashboard (accessible via the shield icon) showing: count of blocked ads, blocked trackers, blocked fingerprinting attempts, and HTTPS upgrade status.

4.3 Agentic AI Assistant (Groq Powered)

- **FR-3.1 DOM Extraction:** **[MUST]** The system shall extract a condensed version of the current tab's interactive elements (Accessibility Tree) to pass context to the AI. The extraction shall handle iframes, shadow DOM trees, and dynamically loaded content. For pages requiring scrolling, the agent shall trigger scroll events to load lazy content before extraction.
- **FR-3.2 Cloud-Based Inference Engine:** **[MUST]** The system shall route all heavy reasoning, DOM evaluation, and NLP tasks to the Groq API. No LLM inference shall run locally on the user's machine.
- **FR-3.3 Primary Reasoning Model:** **[MUST]** The agentic backend shall utilize the most capable available model on the Groq platform (currently **Llama 3.3 70B Versatile**) for complex decision-making, DOM mapping, and multi-step action planning. The model identifier shall be configurable via a settings file to accommodate model deprecations and upgrades without code changes.
- **FR-3.4 Dual-Model Routing:** **[SHOULD]** The system shall route lightweight tasks (search autocomplete, page summarization, simple Q&A) to a smaller, faster model (currently **Llama 3.1 8B Instant**) on Groq. The model identifier shall be configurable. Routing logic shall be determined by task classification heuristics in the local client.
- **FR-3.5 Tool Calling & Structured Output:** **[MUST]** The browser agent shall enforce strict JSON mode in Groq API requests for browser automation commands. The system shall validate the LLM's JSON output against a predefined action schema before

execution. Malformed or schema-violating responses shall be retried up to 2 times before reporting an error to the user.

- **FR-3.6 Action Execution: [MUST]** The agent shall execute state-changing actions on the web based on natural language prompts. The supported action types are:
 - `click(selector)` — Click an element identified by CSS selector or accessibility tree index.
 - `type(selector, text)` — Type text into an input field.
 - `scroll(direction, amount)` — Scroll the page or a specific container.
 - `navigate(url)` — Navigate the current tab to a URL.
 - `select(selector, value)` — Select an option from a dropdown.
 - `wait(ms)` — Pause execution for a specified duration.
 - `extract(selector)` — Extract text content from an element.

The following actions are **not supported** in v1.0: drag-and-drop, file upload, browser alert/dialog interaction.

- **FR-3.7 Conversation Memory: [SHOULD]** The agent shall maintain a session-scoped conversation history to support multi-turn task execution. The history shall include user instructions, agent actions taken, and observed page state changes. The conversation context shall be cleared when the user starts a new task or closes the assistant panel.
- **FR-3.8 Action Transparency & Control: [MUST]** The browser shall display a real-time action log panel showing each step the agent takes, including: the action type, target element description, execution status (pending/running/success/failed), and timestamp. The user shall have the ability to pause, resume, or cancel agent execution at any point.
- **FR-3.9 Offline / Degraded Mode: [MUST]** When the Groq API is unavailable (network failure, rate limiting, API outage), the browser shall: (a) continue functioning as a standard browser with all privacy features operational, (b) display a non-intrusive notification informing the user that AI features are temporarily unavailable, (c) disable the AI assistant panel with a visible retry mechanism, and (d) automatically re-enable AI features when connectivity is restored.

5. Non-Functional Requirements (NFR)

5.1 Performance & Resource Efficiency

- **NFR-1.1 Memory Footprint:** **[MUST]** The core browser shell shall consume no more than 150 MB of RAM while idle (zero tabs open). Each additional open tab shall add no more than 30 MB to the base memory footprint (excluding page-specific content).
- **NFR-1.2 Ad-Block Latency:** **[MUST]** The network filter must evaluate incoming requests against the full filter list in under 2 milliseconds per request.
- **NFR-1.3 Agent Latency Constraint:** **[SHOULD]** The local processing portion of the agentic loop (DOM extraction, serialization, response parsing, action execution) shall complete in under 200 milliseconds. The total end-to-end latency target, including Groq API round-trip, is 800 milliseconds under normal network conditions (<100ms ping to Groq servers).
- **NFR-1.4 Build Time:** **[SHOULD]** A clean build of the browser shell from source must take less than 5 minutes on a reference system (4-core, 3 GHz CPU, 16 GB RAM, SSD storage).
- **NFR-1.5 Startup Time:** **[SHOULD]** The browser shall launch and display a usable window within 3 seconds on the reference hardware. Filter list loading may continue in the background.

5.2 Security & Safety

- **NFR-2.1 Agent Sandboxing:** **[MUST]** The AI agent must operate with strictly scoped privileges, without access to the local file system. File system access, if ever required, shall demand explicit per-session user approval via a system dialog. Access shall be revoked when the session or tab is closed.
- **NFR-2.2 DOM Minimization:** **[MUST]** Before transmitting page context to the Groq API, the local shell must strip all local network details, hidden form tokens (e.g., CSRF tokens), and sensitive input values (passwords, credit card numbers). The system shall maintain a local, append-only audit log of the data payloads sent to the API for user review.
- **NFR-2.3 Explicit Consent:** **[MUST]** The AI agent must require explicit human confirmation before submitting any payment details, passwords, or destructive/irreversible actions (e.g., account deletion, purchase confirmation). The confirmation dialog shall display a human-readable summary of the action the agent intends to perform, including the target URL and affected data.
- **NFR-2.4 API Key Management:** **[MUST]** The Groq API key shall be stored using the OS-native credential manager (Windows Credential Manager, macOS Keychain, Linux

Secret Service) and shall never be written to disk in plaintext, logged, or included in crash reports.

- **NFR-2.5 Content Security Policy: [MUST]** The browser shell's own UI (chrome) shall enforce a strict Content Security Policy to prevent XSS attacks on the browser interface itself. Inline scripts and `eval()` shall be prohibited in the browser chrome.
- **NFR-2.6 Transport Security: [MUST]** All communication between the browser and the Groq API shall use TLS 1.2 or higher. Certificate pinning shall be considered for the Groq API endpoint.

5.3 Reliability & Availability

- **NFR-3.1 Crash Recovery: [MUST]** The browser shall implement session state persistence so that open tabs can be restored after an unexpected crash or forced restart.
- **NFR-3.2 Crash Rate: [SHOULD]** The browser shall have a crash-free session rate of 99.5% or higher under normal usage conditions.
- **NFR-3.3 API Resilience: [MUST]** The system shall implement exponential backoff retry logic for transient Groq API failures (HTTP 429, 500, 502, 503). After 3 consecutive failures, the system shall enter degraded mode (see FR-3.9).

5.4 Maintainability

- **NFR-4.1 Modular Architecture: [SHOULD]** The codebase shall maintain clear separation between the three subsystems (UI shell, Privacy Engine, Agentic Pipeline) with well-defined IPC interfaces, allowing each subsystem to be modified or replaced independently.
- **NFR-4.2 Configuration-Driven: [SHOULD]** All model identifiers, API endpoints, filter list URLs, and tunable thresholds (tab suspension timeout, retry limits, latency targets) shall be externalized to a configuration file rather than hardcoded.
- **NFR-4.3 Logging: [SHOULD]** The application shall implement structured logging (JSON format) with configurable log levels (error, warn, info, debug) for all subsystems.

5.5 Accessibility

- **NFR-5.1 Keyboard Navigation: [SHOULD]** All browser chrome controls (address bar, tabs, settings, AI panel) shall be fully operable via keyboard alone.
- **NFR-5.2 Screen Reader Support: [SHOULD]** All browser chrome UI elements shall have appropriate ARIA labels and roles to support screen reader navigation.

- **NFR-5.3 Color Contrast:** **[SHOULD]** The browser chrome UI shall meet WCAG 2.1 Level AA minimum contrast ratios (4.5:1 for normal text, 3:1 for large text).

6. External Interface Requirements

6.1 Groq API Interface

Property	Specification
Protocol	HTTPS REST API (OpenAI-compatible chat completions format)
Base URL	<code>https://api.groq.com/openai/v1/chat/completions</code>
Authentication	Bearer token via <code>Authorization</code> header
Request Format	JSON with <code>response_format: {"type": "json_object"}</code> for tool calls
Rate Limits	Subject to Groq API tier limits (currently 30 RPM on free tier). The client shall respect <code>Retry-After</code> headers.
Models Used	Primary: <code>llama-3.3-70b-versatile</code> , Secondary: <code>llama-3.1-8b-instant</code> (configurable)

6.2 OS Webview Interface

Platform	Webview Engine	Minimum Version
Windows	WebView2 (Chromium-based)	Windows 10 v1803+
macOS	WebKit (WKWebView)	macOS 11 (Big Sur)+
Linux	WebKitGTK	WebKitGTK 2.36+

6.3 Tauri IPC Interface

Communication between the JavaScript frontend and the Rust backend occurs via Tauri's IPC mechanism:

- **Commands:** The frontend invokes Rust functions via `invoke('command_name', { payload })` . All commands return typed JSON responses.
- **Events:** The Rust backend emits events (e.g., `agent-action-update` , `block-count-update`) that the frontend subscribes to via `listen('event_name', callback)` .

- **Error Handling:** All IPC commands shall return a `Result<T, E>` type, with errors serialized as JSON containing an error code and human-readable message.

7. Data Requirements

7.1 Local Data Storage

Data Type	Storage Mechanism	Encryption	Retention
Bookmarks	SQLite database in Tauri app data directory	None (non-sensitive)	Until user deletes
Browsing History	SQLite database in Tauri app data directory	None (non-sensitive)	Configurable (default: 90 days)
Saved Passwords	OS-native credential manager	OS-managed encryption	Until user deletes
Cookies	Webview-managed cookie store	Webview-managed	Per cookie policy (FR-2.6)
AI Conversation History	In-memory (session-scoped)	N/A (not persisted)	Until session ends
API Audit Log	Append-only log file in app data directory	None (user-controlled)	Configurable (default: 30 days)
Filter Lists Cache	Binary cache file in app data directory	None	Refreshed every 24 hours
Groq API Key	OS-native credential manager	OS-managed encryption	Until user revokes

7.2 Data Sent to External Services

Privacy-Critical: The only external service that receives user data is the Groq API. The following data is transmitted:

- Sanitized accessibility tree of the current page (stripped of sensitive values per NFR-2.2).
- User's natural language instruction to the agent.
- Agent conversation history for the current session (for multi-turn context).

The following data is **never** transmitted:

- Passwords, credit card numbers, or other sensitive form field values.
- CSRF tokens or hidden authentication tokens.
- Local file system paths or network configuration.
- Browsing history, bookmarks, or other stored user data.

8. Error Handling & Fallback Behavior

Error Scenario	Detection	Fallback Behavior
Groq API unreachable (network failure)	Connection timeout (5s) or DNS resolution failure	Enter degraded mode (FR-3.9). Display notification. Retry with exponential backoff (max 3 retries). All non-AI features continue.
Groq API rate limited (HTTP 429)	429 Too Many Requests response	Respect Retry-After header. Queue the request. Inform user of estimated wait time.
Groq API returns malformed JSON	JSON parse failure or schema validation failure	Retry the request up to 2 times with the same prompt. If all retries fail, display error to user and log the malformed response.
Groq API key invalid or expired	401 Unauthorized response	Prompt the user to re-enter their API key via a secure dialog. Do not retry with the same key.
LLM returns unsafe or out-of-scope action	Action schema validation flags unknown action type	Reject the action. Log the event. Inform the user: "The AI attempted an unsupported action and was blocked."
DOM extraction fails (page too complex)	Extraction timeout (>5s) or memory threshold exceeded	Return a truncated accessibility tree (top-level elements only). Warn the user that the agent may have limited context.
Filter list update fails	HTTP error or checksum mismatch during list download	Continue using the cached (stale) filter list. Retry update on next scheduled cycle. Log the failure.
Webview crash	Webview process exit event	Display a "page crashed" placeholder in the affected tab. Offer to reload. Other tabs remain unaffected.

Password decryption failure	OS credential manager returns error	Do not autofill. Log the error. Prompt the user to re-enter the password manually.
-----------------------------	-------------------------------------	--

9. Deployment & Installation

9.1 Target Platforms

Platform	Minimum OS Version	Installer Format	Priority
Windows	Windows 10 v1803 (April 2018 Update)	.msi / .exe (NSIS or WiX)	[MUST]
macOS	macOS 11 (Big Sur)	.dmg	[SHOULD]
Linux	Ubuntu 22.04 / Fedora 38 equivalent	.deb / .ApplImage	[COULD]

9.2 Installation Requirements

- The installer shall not require administrator/root privileges for per-user installation.
- On Windows, the installer shall check for and prompt installation of WebView2 Runtime if not present.
- On Linux, the installer shall verify that WebKitGTK is available and provide clear instructions if missing.
- First launch shall guide the user through initial setup: Groq API key entry, default privacy level selection, and optional data import from another browser.

9.3 Update Strategy

- [SHOULD] The browser shall support auto-update checks on startup (configurable). Updates shall be downloaded in the background and applied on the next restart.
- [SHOULD] The update mechanism shall verify the integrity and authenticity of update packages via digital signature verification.

10. Appendices

Appendix A: Requirement Traceability Matrix

The following table maps each requirement to its corresponding use case and test strategy.

Requirement ID	Requirement Summary	Related Use Case(s)	Test Strategy
FR-1.1	Standard web navigation	UC-1, UC-3	Automated UI tests (navigation, history)
FR-1.2	Multi-tab interface	UC-1, UC-3	Automated UI tests (tab lifecycle)
FR-1.3	Local user data management	UC-2	Unit tests (storage), manual security review
FR-2.1 – FR-2.4	Privacy & ad-blocking engine	UC-1	Unit tests (filter matching), integration tests (webview interception)
FR-3.1 – FR-3.6	Agentic AI core pipeline	UC-2, UC-3, UC-5	Integration tests (mock API), end-to-end tests (live API)
FR-3.8	Action transparency & control	UC-2, UC-3	Manual UI review, automated panel state tests
FR-3.9	Offline / degraded mode	UC-4	Integration test (disconnect API, verify graceful degradation)
NFR-2.1 – NFR-2.6	Security & safety	All	Security audit, penetration testing, manual code review

Appendix B: Action Schema (FR-3.5 / FR-3.6)

The following JSON schema defines the expected structure of LLM-generated browser actions:

```
{
  "type": "object",
  "required": ["action", "params"],
  "properties": {
    "action": {
      "type": "string",
      "enum": ["click", "type", "scroll", "navigate", "select", "wait", "extract"]
    },
    "params": {
      "type": "object",
      "properties": {
        "selector": { "type": "string" },
        "text": { "type": "string" },

```

```
    "url": { "type": "string", "format": "uri" },
    "direction": { "type": "string", "enum": ["up", "down", "left", "right"] },
    "amount": { "type": "integer", "minimum": 0 },
    "value": { "type": "string" },
    "ms": { "type": "integer", "minimum": 0, "maximum": 10000 }
  },
  "reasoning": {
    "type": "string",
    "description": "The LLM's explanation for why this action was chosen."
  }
}
```